TUTORIAL 1: CAPTURA DE PROJETO E SIMULAÇÃO DE HARDWARE (XILINX ISIM)

CRIANDO UM AMBIENTE DE EXECUÇÃO

- **Hoje, usar localmente a infraestrutura da PUCRS não é uma opção viável na quantidade de vezes que a disciplina requer**
- **Há várias opções para ter acesso ao ambiente Vivado, por exemplo**
  1. Instalar um software de acesso a servidores da PUCRS (Linux)
     - ✓ O software **xpra** é uma excelente opção, gratuita e poderosa
  2. Instalar uma máquina virtual Linux contendo o VIVADO
     - ✓ Funciona bem mas precisa de uma boa máquina e muito disco
  3. Instalar o VIVADO no Windows
     - ✓ Forma simples, mas requer bastante disco/SSD ~45Gbytes

- **Reflitam sobre a disponibilidade individual de cada um de vocês, escolham uma opção, o professor pode ajudar aqui!! Instalem uma das soluções em máquina que tenham à disposição de vocês**
- **Minha recomendação: Instalar Vivado localmente, veja como aqui!**

USANDO O SIMULADOR DO VIVADO (V2020.2)

Criando os arquivos do projeto

1. Criar um diretório de trabalho. Exemplo half_adder
2. Criar os dois arquivos fonte, com extensão VHDL

✓ somador1.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;

entity halfadd is
port (A, B: in std_logic;
      Sum, Carry: out std_logic);
end halfadd;

architecture comp of halfadd is
begin
  Sum   <= A xor B;
  Carry <= A and B;
end comp;
```

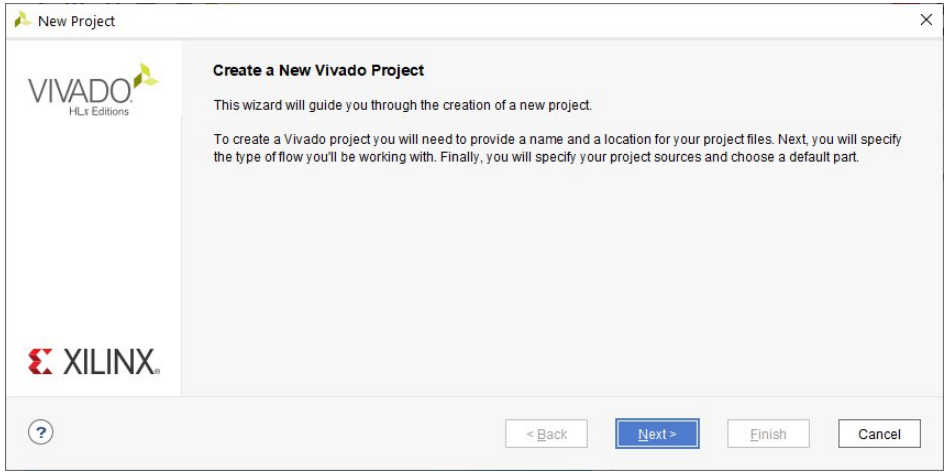
✓ somador1_tb.vhd

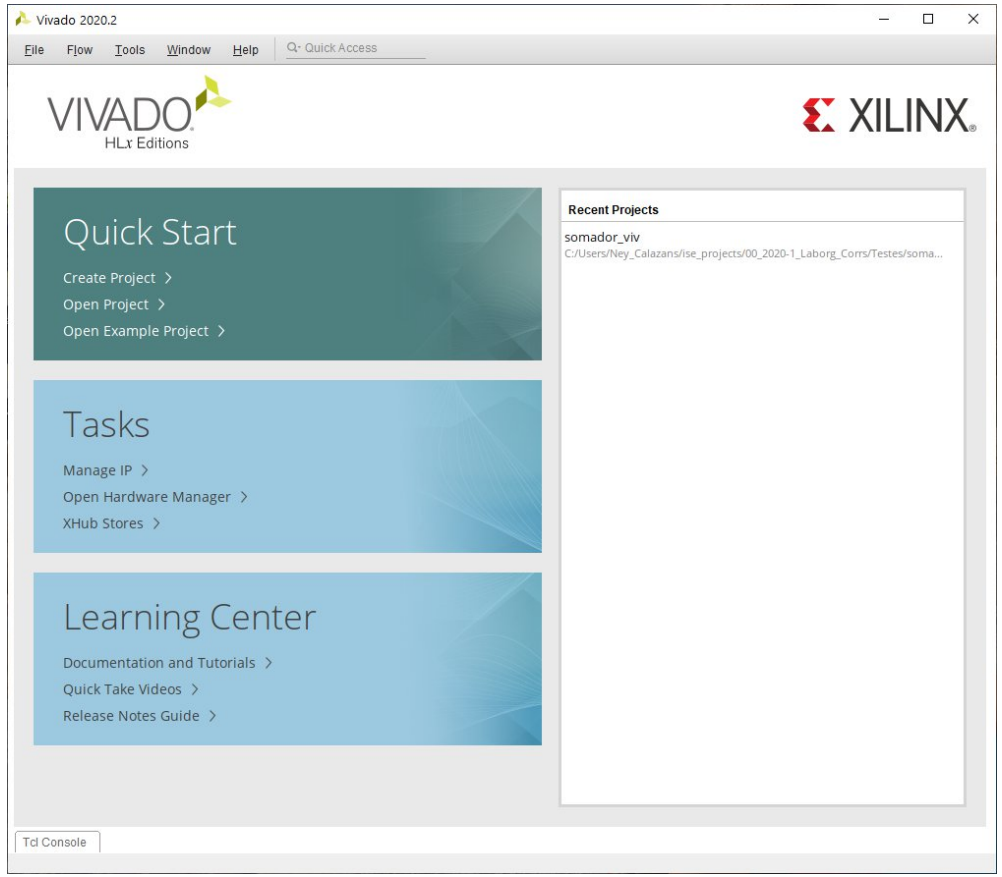
```
library IEEE;
use IEEE.std_logic_1164.all;

entity halfadd_tb is
end halfadd_tb;

architecture TB_ARCHITECTURE of halfadd_tb is
  signal aa, bb, soma, vaium : std_logic;
begin

  UUT : entity work.halfadd
        port map ( A => aa, B => bb,
                   Sum => soma, Carry => vaium );

  aa <= '0', '1' after 10 ns, '0' after 20 ns, '1' after 30 ns;
  bb <= '0', '1' after 20 ns;

end TB_ARCHITECTURE;
```

USANDO O SIMULADOR DO VIVADO (V2020.2)

➢ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

3. Caso algum projeto seja aberto ao lançar o VIVADO, feche-o com a opção de menu File→Close Project
4. Crie um novo projeto de nome somador1 apertando o botão Create Project. (Lembre-se: escolha um diretório no qual tens direito de escrita). Leia o conteúdo da tela inicial e depois clique no botão Next.

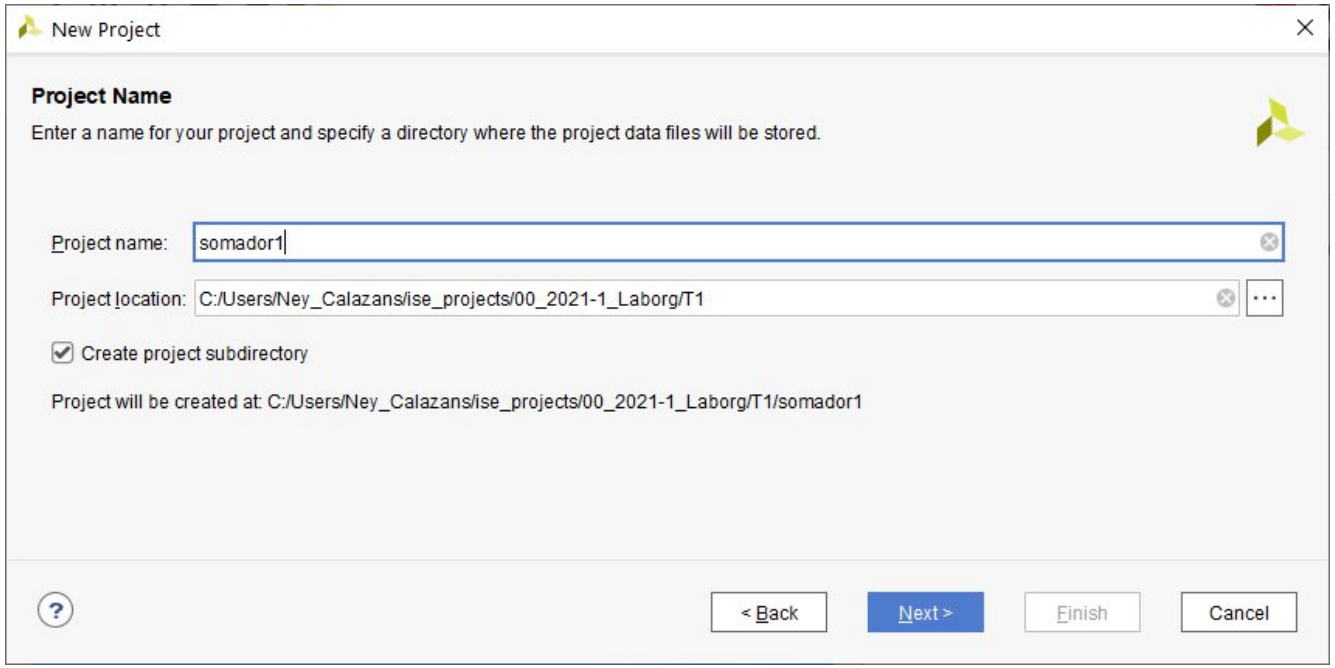




USANDO O SIMULADOR DO VIVADO (V2020.2)

➢ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

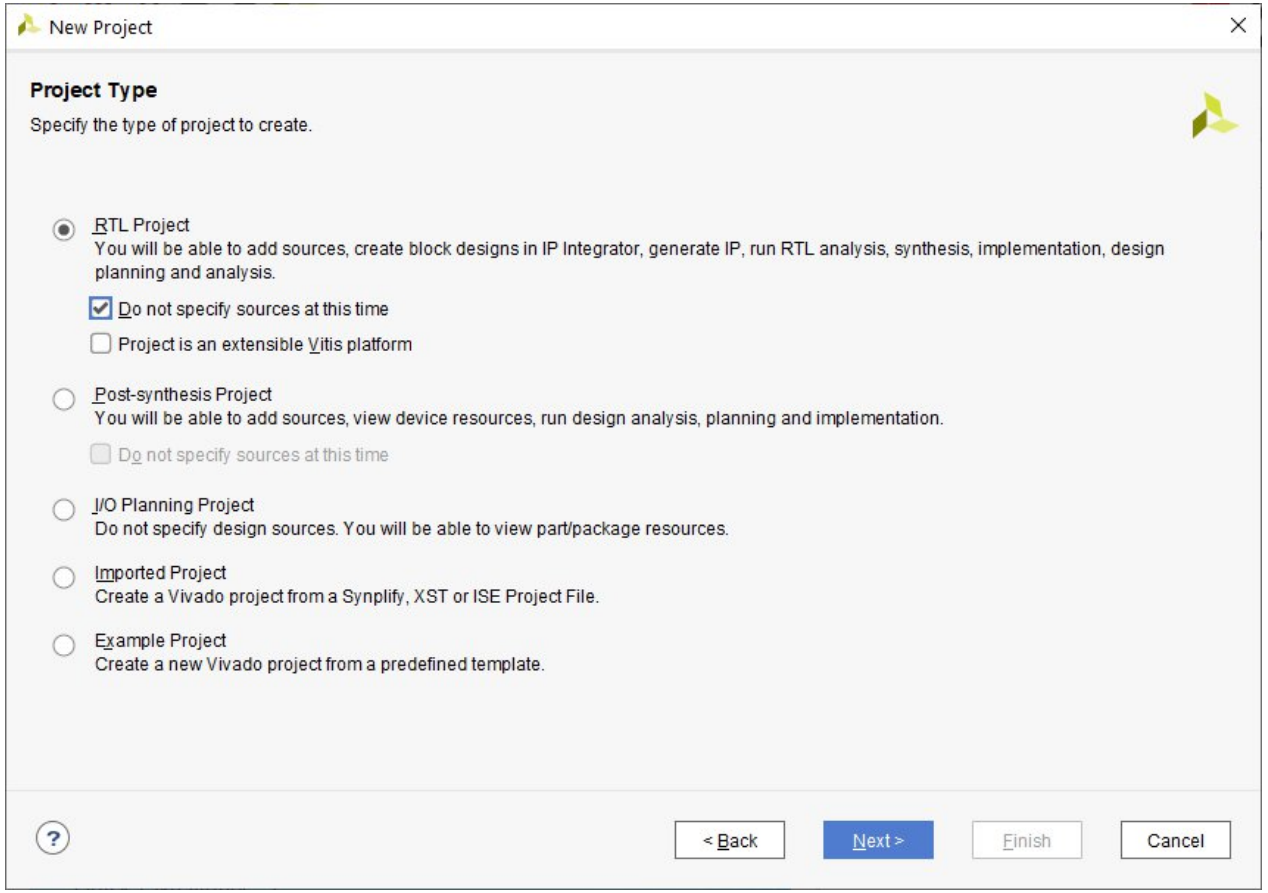
5. Preencha os campos Project name e Project location. Observação: Não usem caracteres especiais (acentos, brancos, etc.), seja no nome do projeto, seja no nome de qualquer parte do caminho. Garanta que a caixa ao lado de Create project subdirectory esteja marcada. Clique no botão Next.



USANDO O SIMULADOR DO VIVADO (V2020.2)

➢ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

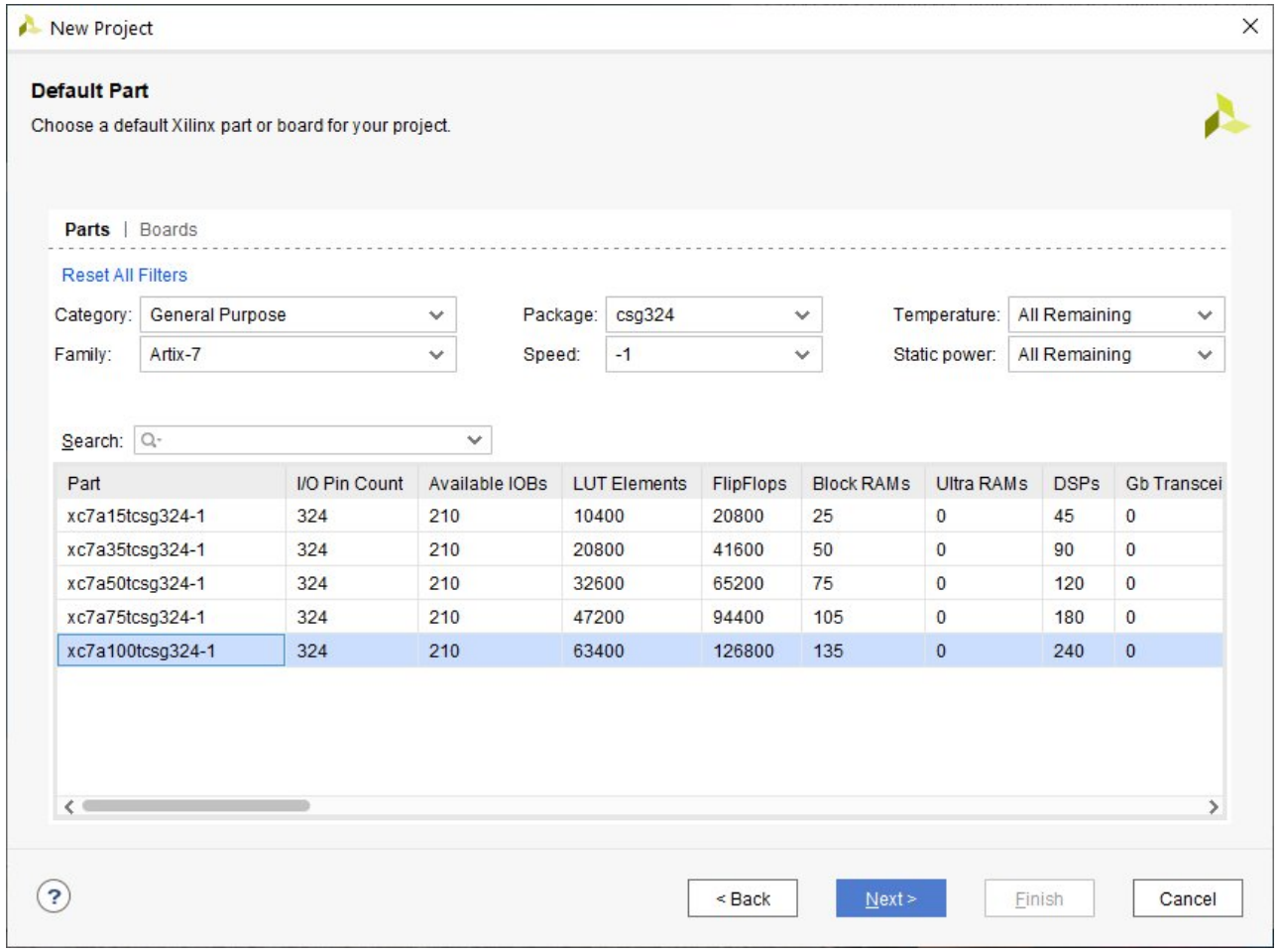
6. Na janela seguinte, escolha a opção RTL Project. RTL significa *Register Transfer Level*, ou Nível de Transferência entre Registradores o nível de abstração em que costumamos descrever hardware ao usar VHDL para descrever um circuito que queremos construir. Marque e caixa ao lado da opção Do not specify sources at this thime, pois vamos acrescentar manualmente os arquivos fonte do projeto. Em seguida, clique no botão Next.

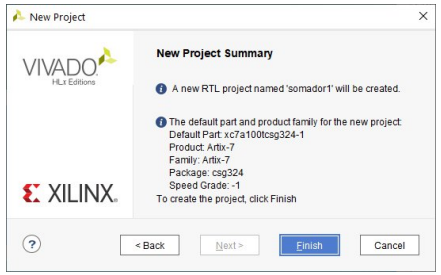


USANDO O SIMULADOR DO VIVADO (V2020.2)

➢ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

7. Na janela seguinte, faça as escolhas marcadas na Figura ao lado para achar o dispositivo da placa Nexys A7. Estas são Category: General Purpose; Family: Artix-7; Package: csg324; Speed: -1 . Em seguida, clique no botão Next.
8. Confira os conteúdos da janela seguinte (New Project Summary) e depois clique no botão Finish.

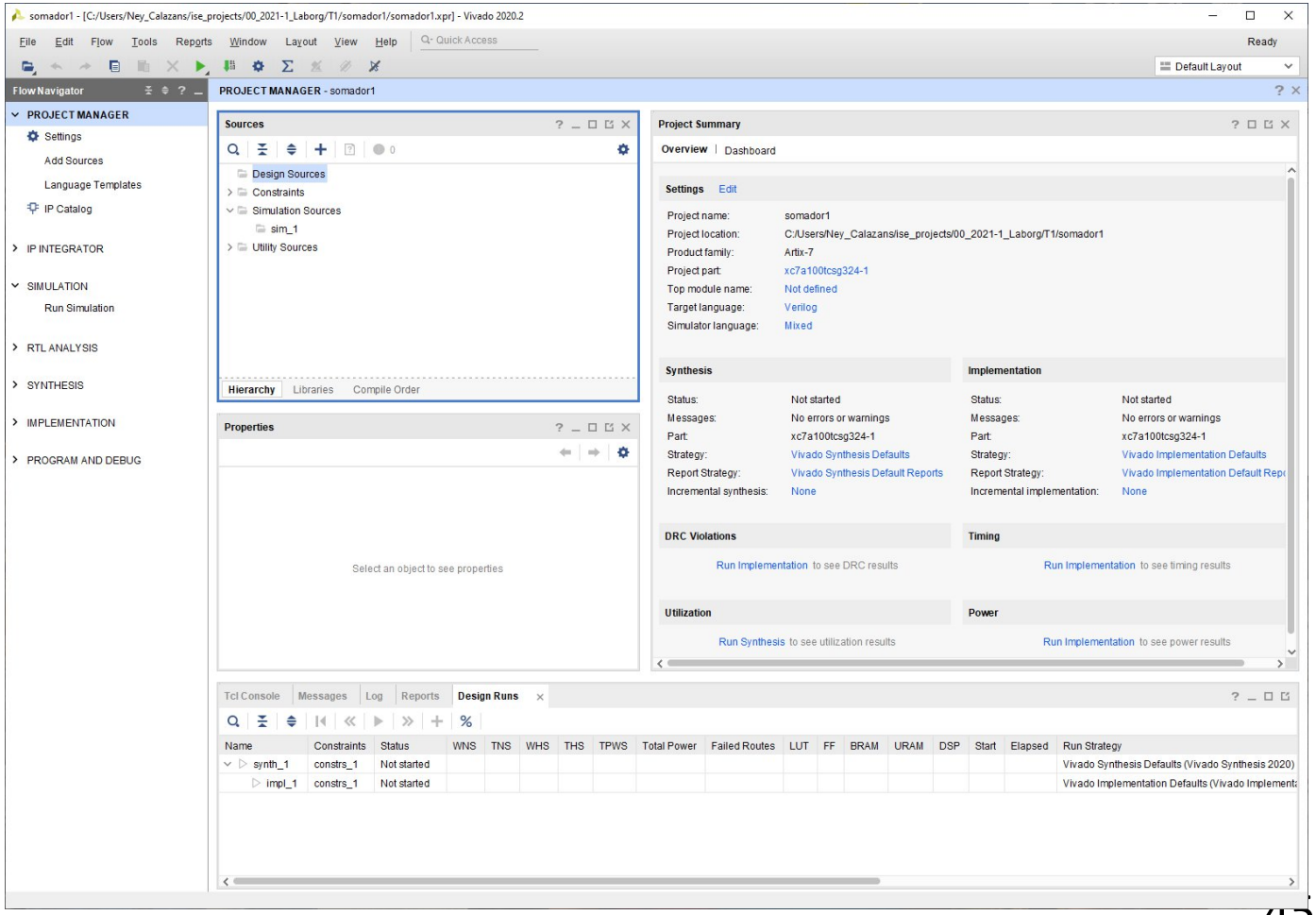


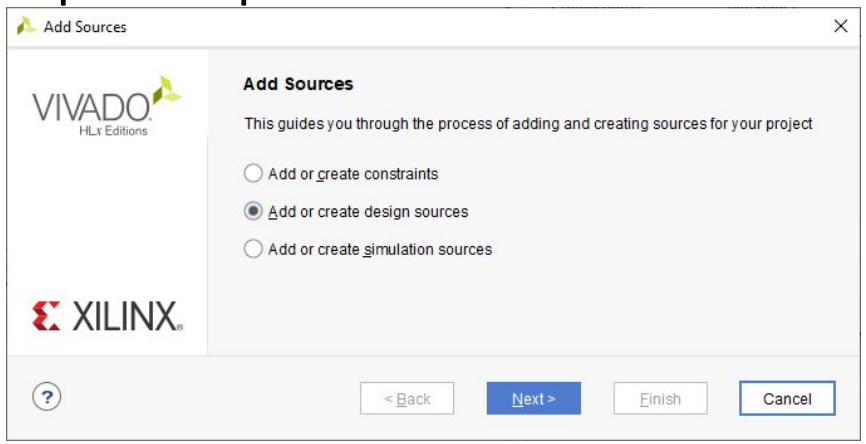


USANDO O SIMULADOR DO VIVADO (V2020.2)

➢ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

9. A janela ao lado deve ser similar ao que se vê. Na janela denominada Sources clique no símbolo + para acrescentar os arquivos fonte ao projeto.
10. Escolha a opção Add or create design sources na janela que aparece e depois clique em Next.

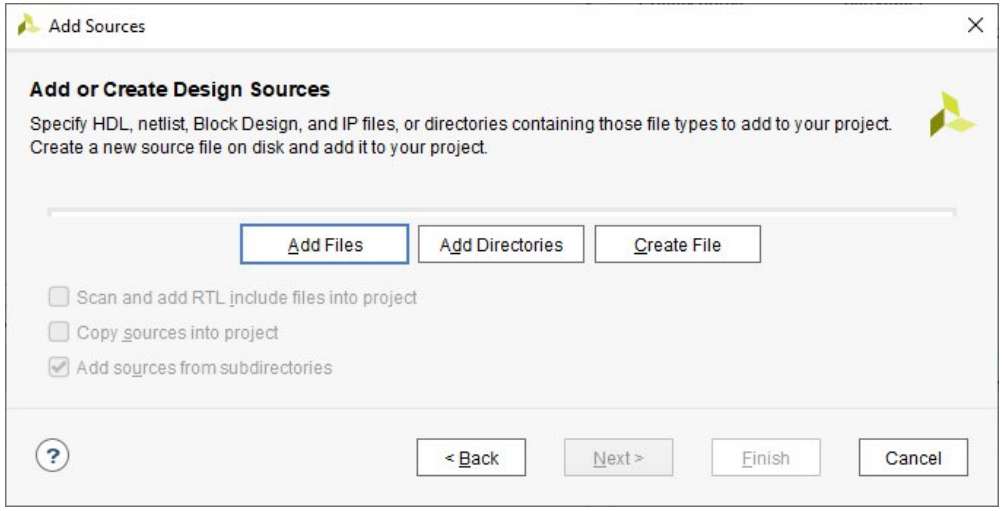


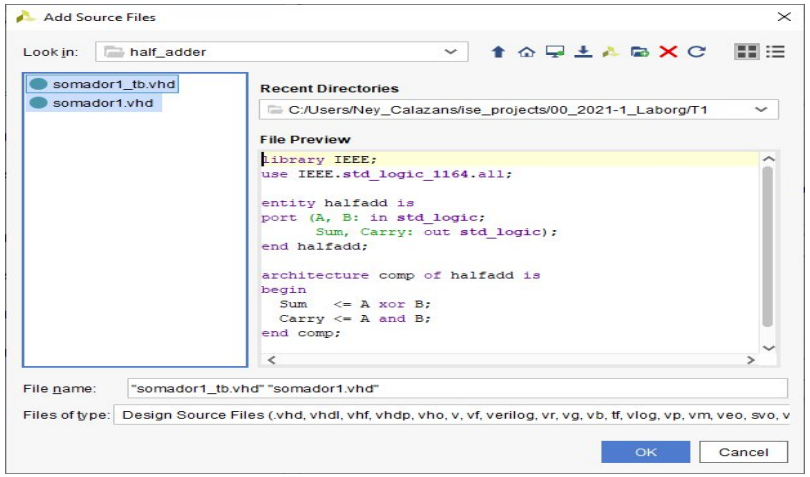


USANDO O SIMULADOR DO VIVADO (V2020.2)

➢ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

11. Na janela Add Sources clique em Add Files.
12. Na janela que surge, navegue até o diretório onde você criou os arquivos somador1.vhd e somador1_tb.vhd, selecione ambos e em seguida clique em OK.
13. Depois, ao voltar à janela Add Sources clique em Finish, que deve agora estar habilitado.

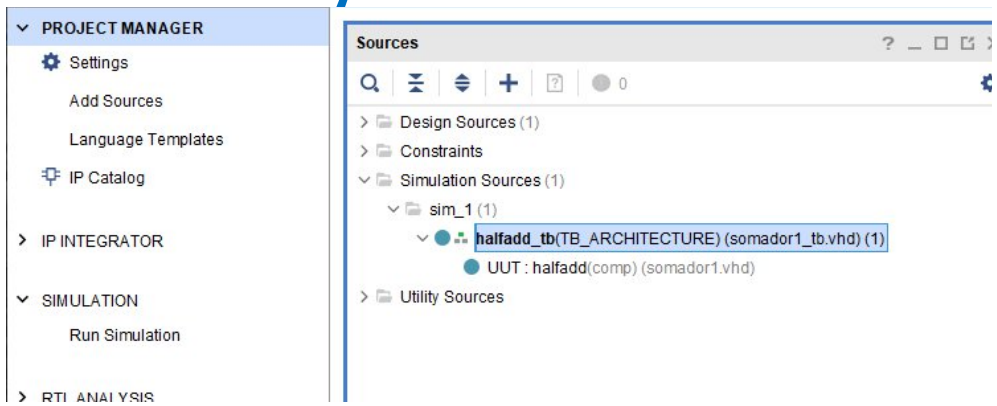


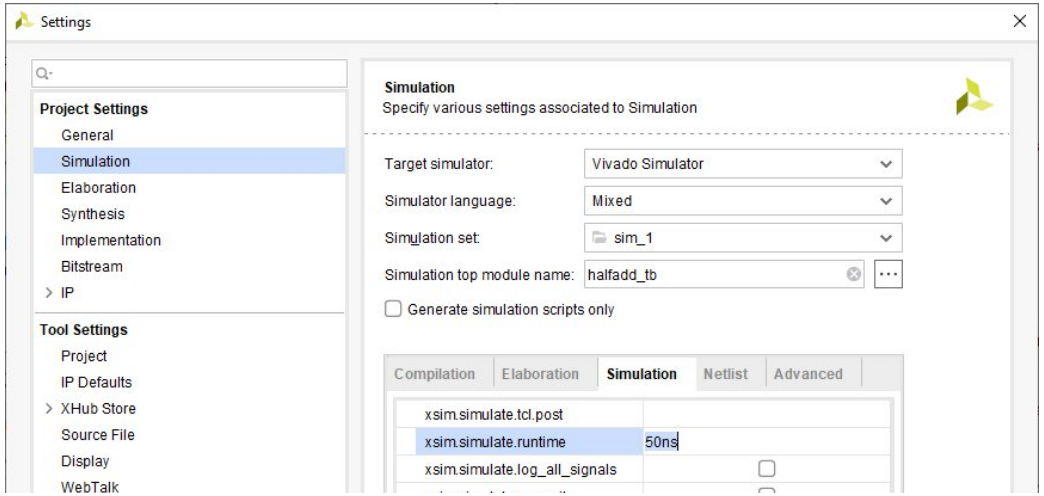


USANDO O SIMULADOR DO VIVADO (V2020.2)

➢ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

14. Na janela Sources, ache agora a entidade halfadd_tb (Sob o diretório Simulation Sources (1) e neste sob o sub-diretório sim_1(1)) e selecione ela.
15. Agora vamos configurar a simulação. Clique com o botão direito do mouse sobre o item Run Simulation sob a opção SIMULATION da janela PROJECT MANAGER. Em seguida escolha a opção Simulation Settings no menu *pop-up* que surge. Na janela que surge, selecione Simulation em Project Settings, e selecione a aba Simulation na janela da direita (Simulation). Na linha xsim.simulate.runtime troque 1000ns (ou o que lá houver) por 50ns. Finalmente, clique em Apply e OK.

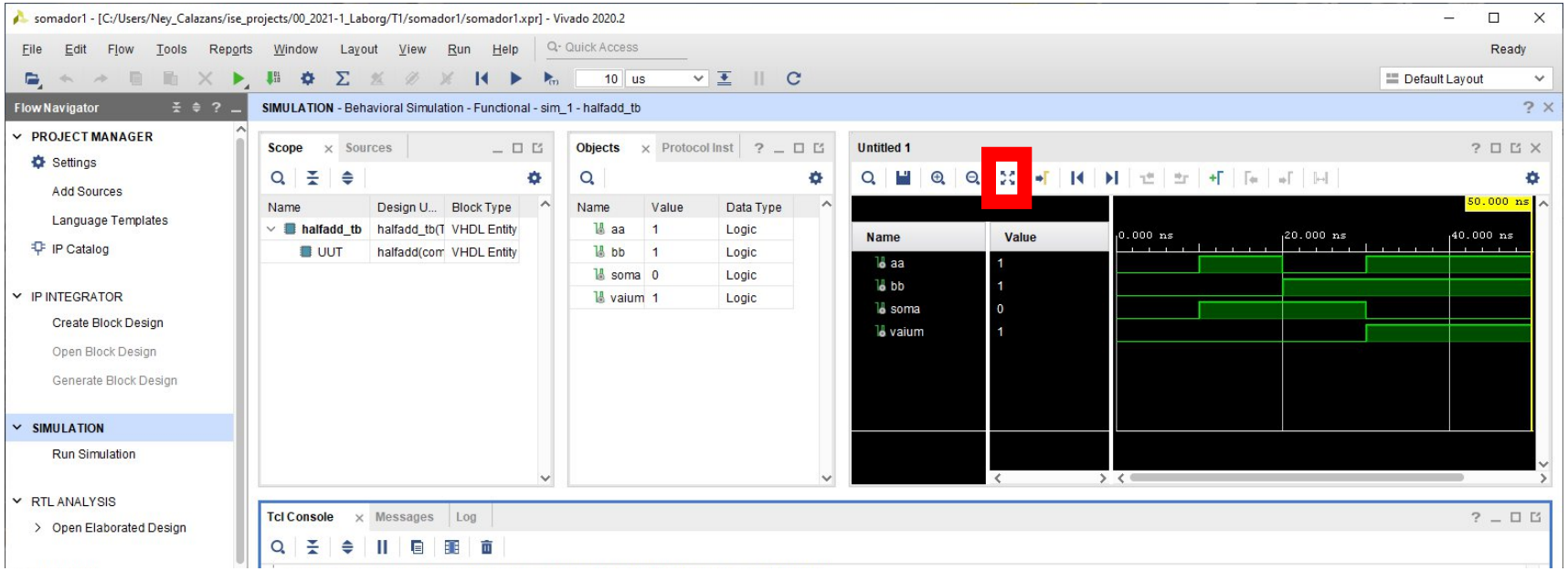




USANDO O SIMULADOR DO VIVADO (V2020.2)

➢ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

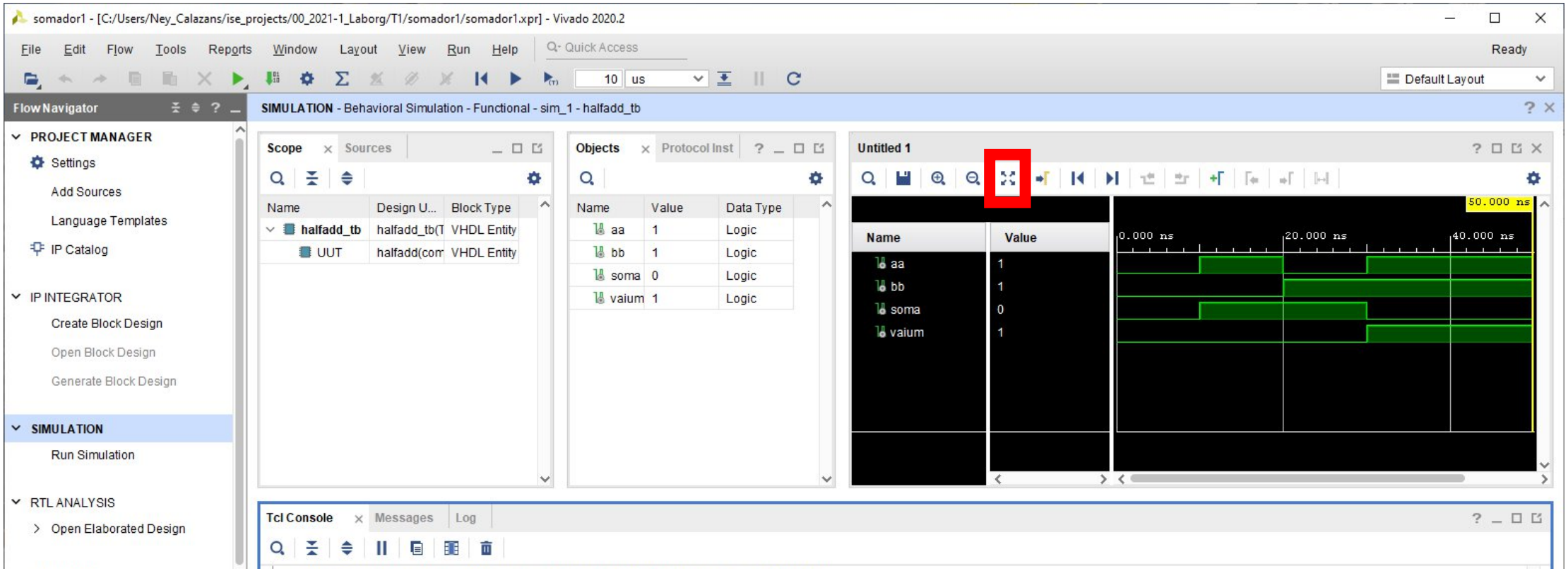
16. Agora, clique na janela PROJECT MANAGER no item Run Simulation sob a opção SIMULATION (desta vez com o **botão esquerdo** do mouse!). No diálogo de pop-up que surge, escolha a opção Run Behavioral Simulation (deve ser a única habilitada).
17. Para ver as formas de onda completas, clique no símbolo com 4 setas diagonais saindo do centro de um quadrado. Veja o resultado esperado abaixo.



USANDO O SIMULADOR DO VIVADO (V2020.2)

<u>Note:</u> Esta forma de onda mostra todas as linhas da tabela verdade do somador (4 combinações de entradas)

<u>Observação:</u> Lembrem-se de simular o testbench e não o projeto, selecionando o arquivo no topo da hierarquia. Para ver toda uma onda simulada na janela clique no botão "Zoom Fit"



TRABALHO T1

TRABALHO T1

- O **Trabalho T1** consiste em executar a simulação de 3 projetos no simulador do ambiente VIVADO da Xilinx
  - ✓ Projeto 1
    - Adaptação ao ambiente de simulação através do desenvolvimento de um meio-somador de 1 bit (half_adder). Este projeto já foi desenvolvido nos slides anteriores (Seção "Usando o Simulador do Ambiente VIVADO da Xilinx") → apenas repita o processo descrito aqui anteriormente, para fixar o processo
  - ✓ Projeto 2
    - Desenvolvimento de um somador de 4 bits sem vai-um (full_adder) → simples
  - ✓ Projeto 3
    - Desenvolvimento de três implementações (arquiteturas) de um flip-flop mestre-escravo assíncrono (ffd) → exigente

PROJETO 2

- Escreva um novo código (faça um novo projeto VHDL no VIVADO. Use a opção *Copy Project* do VIVADO) e utilize sinais não de 1 bit mas de 4 bits (std_logic_vector(3 downto 0)). O efeito é transformar o meio somador de 1 bit em um somador de 4 bits.
  - ✓ Note: std_logic→1 fio e std_logic_vector→1 ou mais fios (barramento)
- A nova entidade VHDL deve parecer com o seguinte

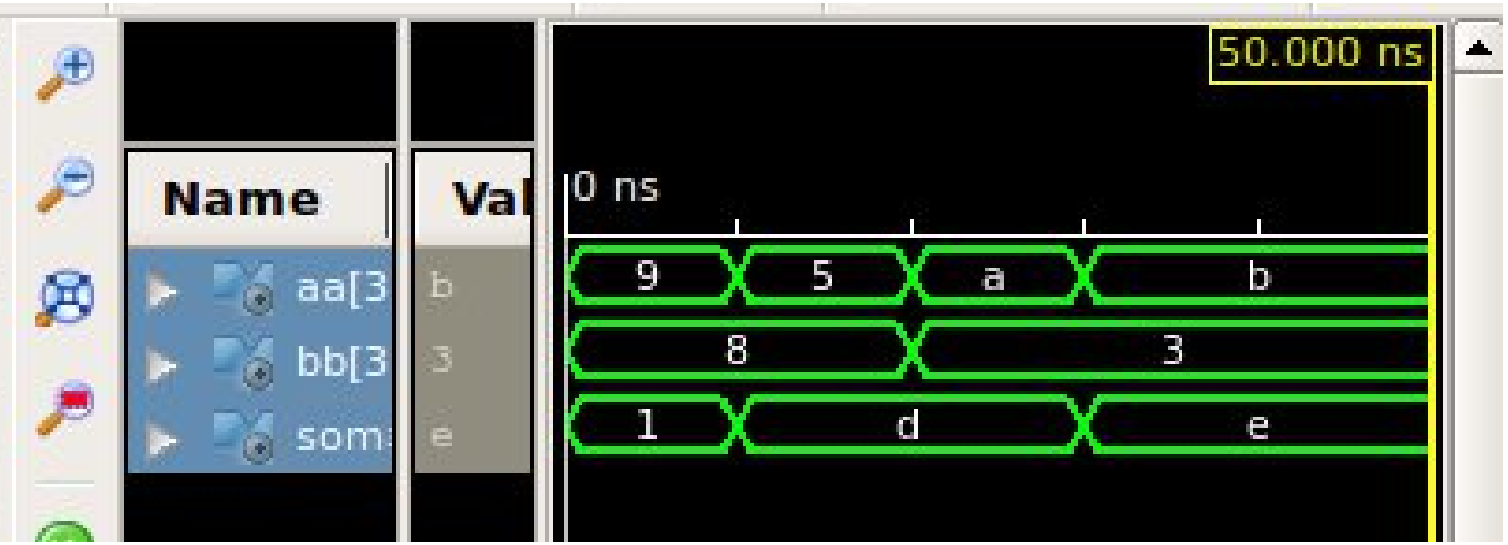
```
entity soma is
port (A, B: in std_logic_vector(3 downto 0);
      Soma: out std_logic_vector(3 downto 0));
end soma;
```

- O corpo da arquitetura VHDL deverá ser alterado e conter apenas "`Soma <= A + B;`"
- Deve-se incluir a linha: use IEEE.std_logic_unsigned.all (Porquê?)
- Escreva o *testbench* correspondente, estude como!

PROJETO 2

- Os 4 bits podem ser representados como um único dígito em hexadecimal. No *testbench* pode-se escrever
  - ✓ `aa <= x"9", x"5" after 10 ns, x"A" after 20 ns, x"B" after 30 ns;`
  - ✓ `bb <= x"8", x"3" after 20 ns;`

- Ao simular este circuito, deve aparecer uma forma de onda similar à apresentada na Figura à direita. Você pode testar somas de outros valores no seu *testbench*



- O comportamento é o esperado? Explique, analisando cada combinação!! Duvide!!

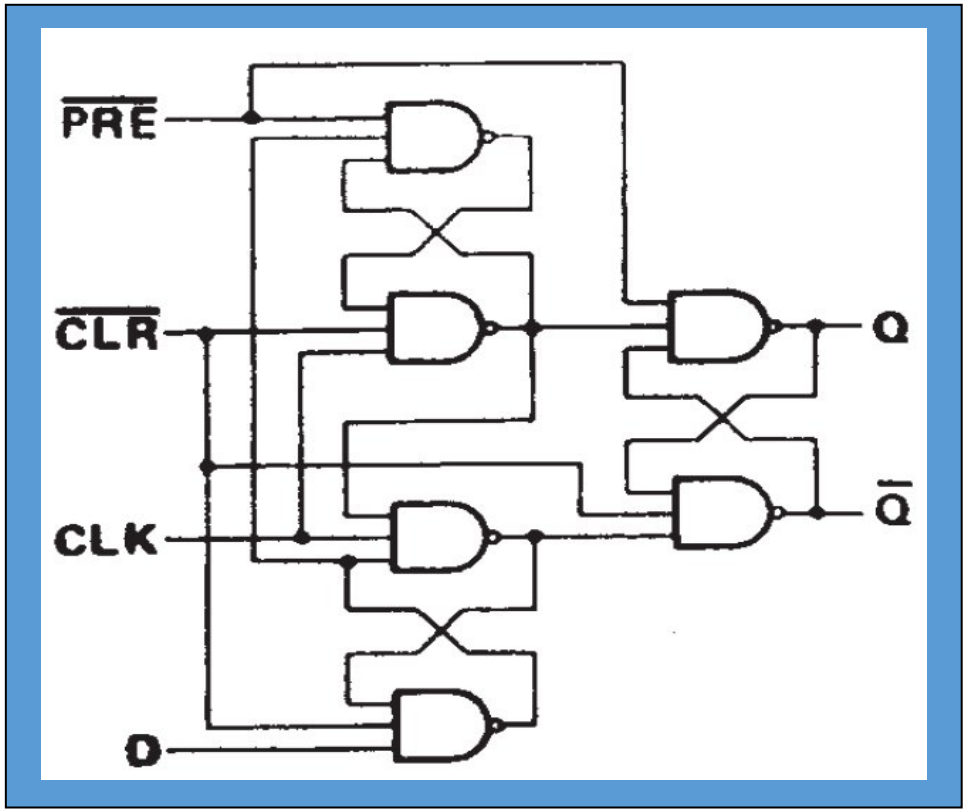
PROJETO 2

- Para gerar todas as combinações possíveis de entradas deste circuito seria necessário produzir 256 padrões (combinando os 16 valores distintos de A com os 16 valores distintos de B). Um trecho de código VHDL capaz de gerar estes estímulos é dado abaixo. Estude-o e entenda-o, simule-o se quiser

```
signal aa : std_logic_vector(3 downto 0):="0000"; -- valor inicial
signal bb : std_logic_vector(3 downto 0):="0000"; -- valor inicial
...
process (aa)
begin
  if (aa/=x"F") then
    aa <= aa+x"1" after 10ns;  -- a cada 10ns
  else aa <= x"0" after 10ns;  -- incrementa A
  end if;
end process;
process (bb)
begin
  if (bb/=x"F") then
    bb <= bb+x"1" after 160ns; -- a cada 16
  else bb <= x"0" after 160ns; -- valores de A,
  end if;                      -- incrementa B
end process;
```

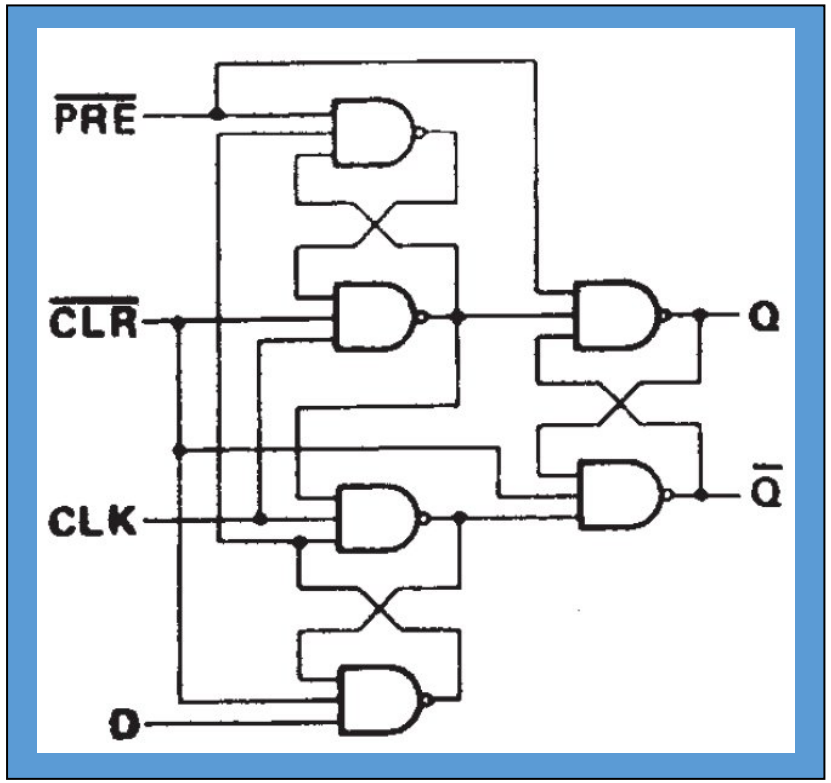
PROJETO 3

1. Implemente o circuito de seis portas lógicas ao lado em VHDL – aqui **é proibido usar o comando process de VHDL!** Este último comando será tema de aula posterior
2. Crie um projeto no ambiente VIVADO que vai conter a entidade (**entity**) correspondente a este circuito e três arquiteturas (**architecture**) descritas mais adiante
3. Crie um *testbench* para cada versão (**architecture**) do circuito. Este deve instanciar cada circuito, gerar estímulos para todas suas 4 entradas e aplicar os estímulos a instância da arquitetura em questão
4. Simule cada *testbench* e descreva textualmente como o circuito funciona
5. Importante: A descrição VHDL deve refletir exatamente a estrutura do diagrama de portas ao lado, não apenas seu comportamento! Por isto a proibição de usar **process**, que serve para descrever comportamento, não estrutura.



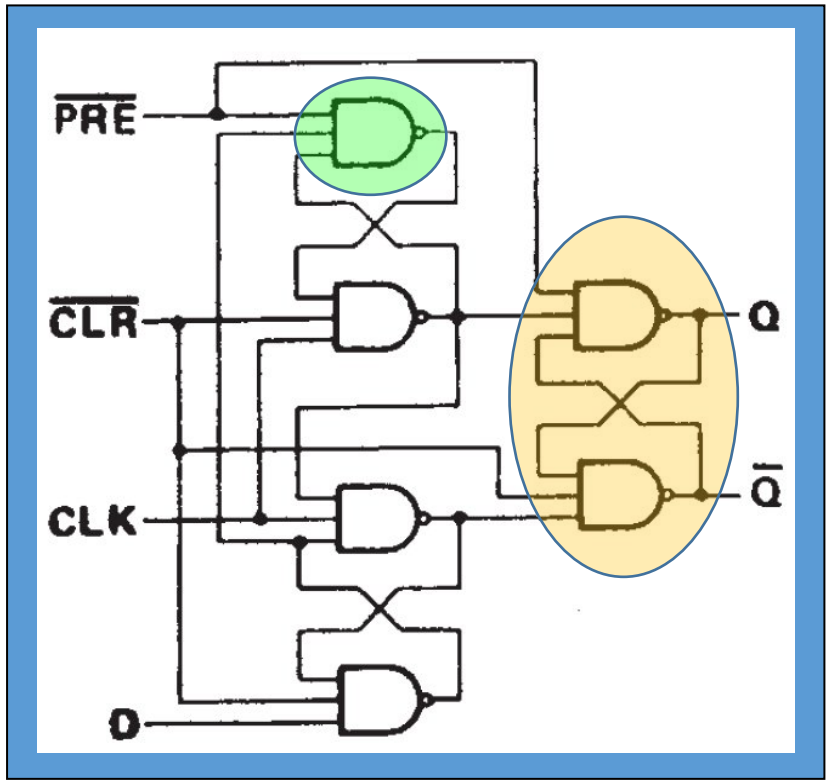
PROJETO 3 – A Primeira Implementação (`architecture`)

1. Observando o diagrama original, nota-se que o circuito usa apenas portas NAND de 3 entradas (digamos uma **NAND3i**)
   - Há seis portas **NAND3i** interconectadas
   - O circuito possui 4 pinos/fios de entrada, 2 pinos/fios de saída e 4 fios internos (**confira este número!!!)**
2. Com base nas observações acima crie uma descrição onde cada porta **NAND3i** é descrita em uma linha de código VHDL. Veja como se descreve uma porta lógica no projeto somador1.
   - Problema: uma NAND de 3 entradas não pode ser descrita estendendo uma NAND de 2 entradas em VHDL. Porquê? Qual a solução?
3. Necessidades
   - Declarar os fios internos
   - Os pinos de saída (Q e Q barra) são usados como entrada de portas lógicas, mas VHDL proíbe usar um pino de saída como entrada no mesmo circuito. Como resolver?

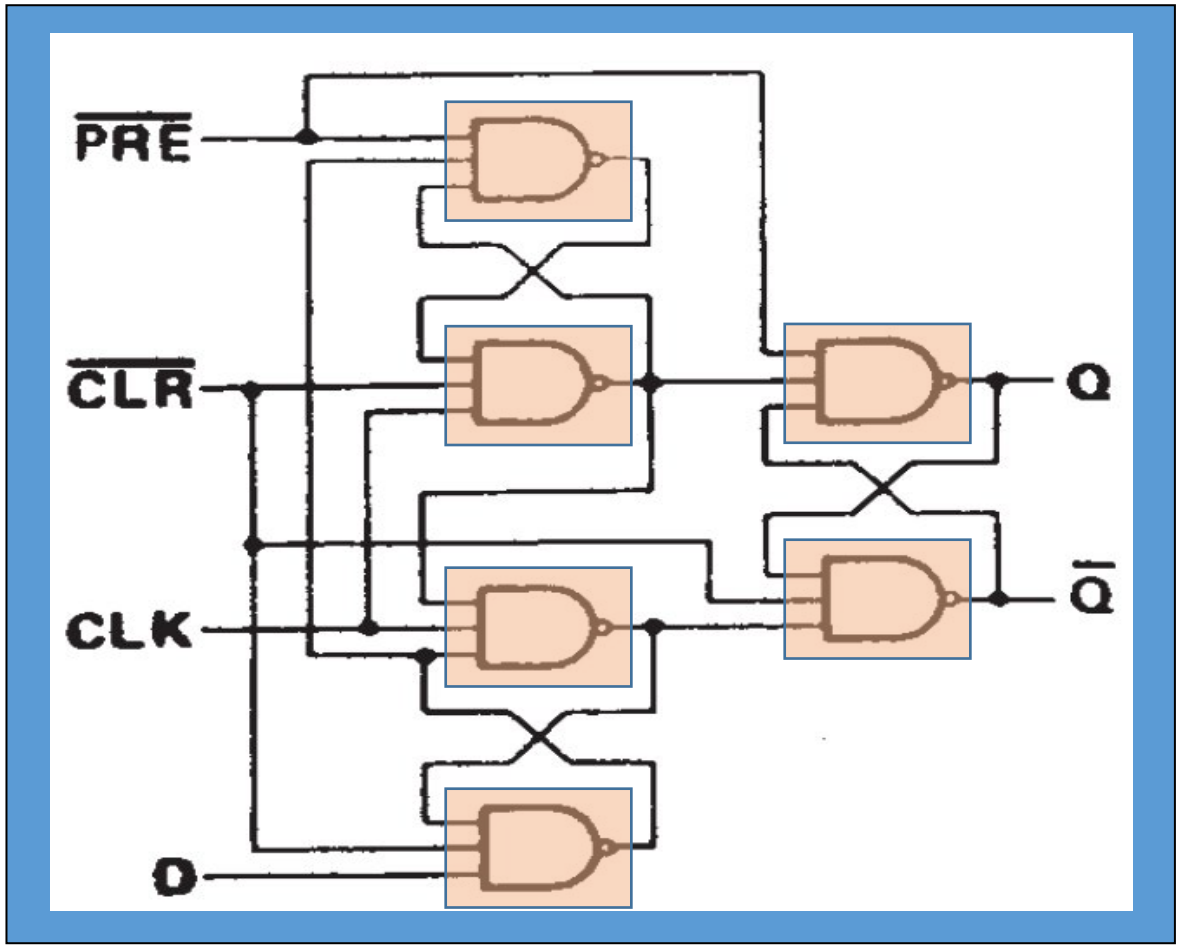


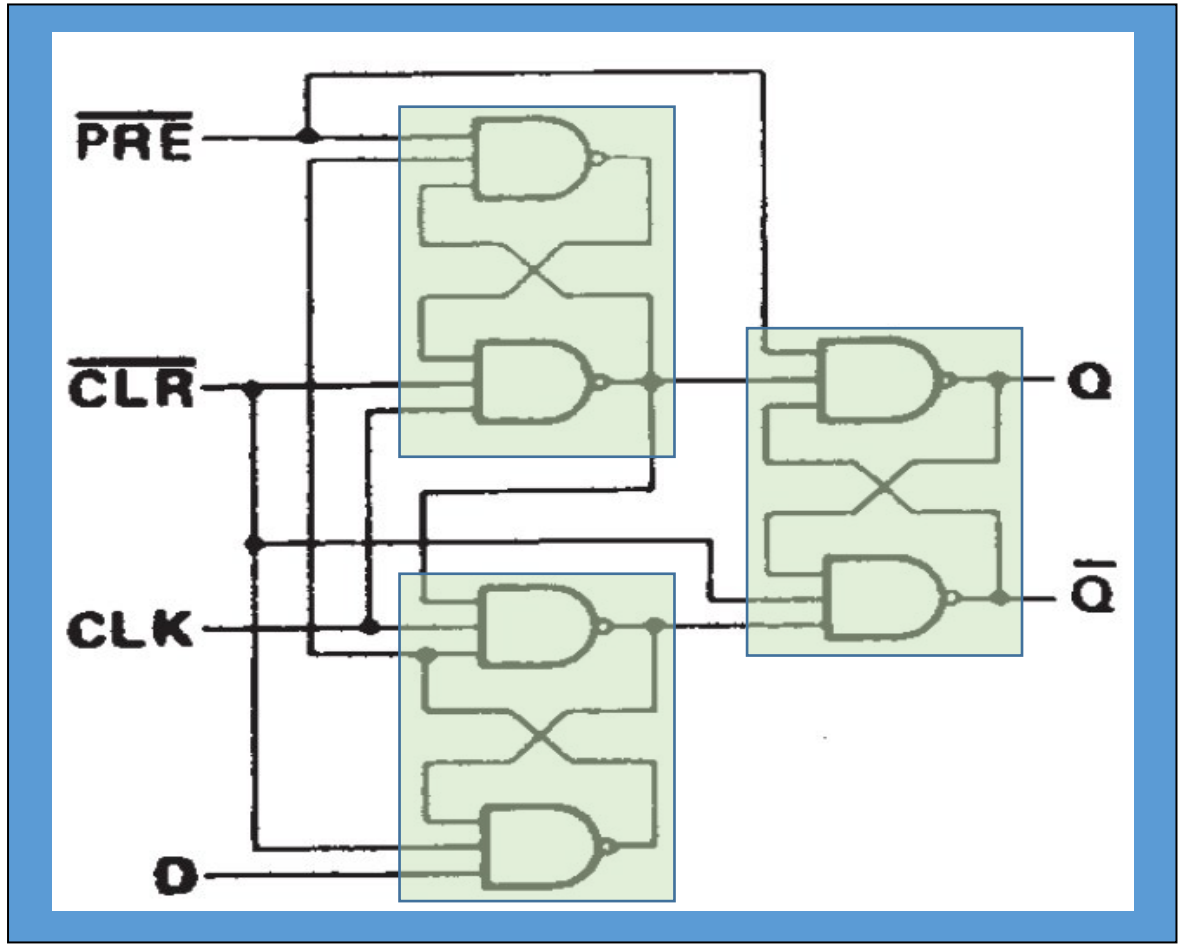
PROJETO 3 – As Implementações (`architecture`) Alternativas

1. Observando novamente o diagrama original, nota-se que existem estruturas similares repetidas no desenho
   - Como dito, o circuito só usa portas **NAND3i**
   - Há três pares de **NAND3i** interconectados do mesmo jeito (saída de uma NAND é entrada da outra e vice-versa, formando um *latch*)
2. Com base nas observações acima crie duas novas versões deste circuito em VHDL, que devem ser **hierárquicas** (??)
   - Uma usando 6 vezes uma **NAND3i** como par entidade/arquitetura
   - Uma usando 3 vezes um *latch* com **4 entradas e 2 saídas** como par entidade/arquitetura (porque 4 entradas e 2 saídas? Justifique!)
3. Estas duas novas arquiteturas lidam com descrições **modulares** e **hierárquicas** em VHDL, usando o comando **port map** de forma extensiva. Estude e aprenda como usar ele!!



PROJETO 3 – As Implementações (`architecture`) Alternativas





Estruturas de implementação alternativas com E/A NAND3i e latch_4ctrls

PROJETO 3

- Informações adicionais
  - O circuito do Projeto 3 é um flip-flop tipo D sensível à borda de subida do relógio (sinal CLK), com sinais assíncronos (isto é independentes do relógio, e prioritários em relação ao sinal de relógio) de preset (sinal $\overline{PRE}$) e reset (sinal $\overline{CLR}$). Estes sinais são **ativos** (ou seja agem) quando valem 0. Equivale a metade de um CI TTL 7474, veja o datasheet dele
  - As saídas são os sinais Q e $\overline{Q}$, tipicamente assumindo valores opostos, e mostram o valor do bit armazenado no FF e seu complemento. Use uma convenção Q, Q_n em VHDL
  - O sinal PRE em 0 força a saída Q para 1 (e $\overline{Q}$ para 0), enquanto CLR em 0 força Q para 0 (e $\overline{Q}$ para 1). Eles não devem nunca ser 0 ao mesmo tempo. Qualquer um em 0 faz com que as bordas do relógio sejam ignoradas (impede o efeito deste sinal de controle)
  - Seguindo a regra geral de circuitos síncronos, nenhuma entrada deve mudar ao mesmo tempo que o relógio. Porquê???
  - Cada borda de subida de CLK sem PRE nem CLR ativados provoca a amostragem e o armazenamento do valor de D → funcionamento normal!!

PROJETO 3

- Notem que o circuito a implementar é sequencial e assíncrono (devido aos laços de realimentação combinacionais), e cria um componente básico para projetos síncronos. Como já foi mencionado, trata-se de um flip-flop D sensível à borda de subida do clock

- Cuidados a tomar
  - ✓ Ao escrever o *testbench*, respeitem a relação de temporização entre sinais de um circuito síncrono
    - Quando o clock mudar na sua borda sensível (0→1, ou borda de subida), NENHUM sinal amostrado por ele (D, neste caso) pode mudar ao mesmo tempo! Lembrem-se dos conceitos de tempo de *setup* e *hold*! Se lembram???
    - Os sinais CLR e PRE também devem respeitar esta regra!
  - ✓ **Notem, nomes de sinais no *testbench* podem ser idênticos aos nomes dos pinos dos objetos que o *testbench* instancia. Convenção: CLR pode receber nome CLR_n. Esta convenção pode (deve) ser usada para todos os sinais ativos em 0 (com barra sobre o nome)**

RESUMO DO TRABALHO T1

- No Projeto 3, existem três arquiteturas (**architecture**) mas apenas uma entidade (**entity**) Estude, aprenda e use o conceito VHDL de **configuração** (**configuration**) para simular cada uma das arquiteturas que compartilham a mesma entidade
- O **Trabalho T1** compreende ao todo 3 projetos VIVADO
  - ✓ Projeto 1 - Meio-somador de 1 bit (somador1) (1 ponto)
  - ✓ Projeto 2 - Somador de 4 bits sem vai-um (full_adder) (1 ponto)
  - ✓ Projeto 3 - Flip-flop D sensível à borda do clock (ffd) – com três arquiteturas alternativas (6 pontos)
- O que deve ser entregue
  - ✓ Arquivo compactado (formato **ZIP** ou equivalente): <aluno1-aluno2>.zip, com os seguintes **arquivos**
    - ✓ Um relatório com texto, trechos de código e formas de onda, mostrando com os circuitos operam e sua estrutura (2 pontos)
    - ✓ Para cada 1 dos 3 projetos, anexe somente o código VHDL de cada implementação (três no caso do terceiro projeto) e os testbenches que validam cada implementação
      - Por exemplo `half_adder/half.vhd, half_adder/tb_half.vhd, full_adder/full.vhd ...`